Ft_Printf

Project description

Replicate the printf function. The functions in the printf family produce output according to a format. Uses <stdio.h> library

Allowed functions: malloc, free, write, va_start, va_arg, va_copy, va_end

+

Libft

Prototype

```
int   ft_printf(const char *, ...)
```

Usage:

```
printf("hello my name is %s and I'm %d years old", "M", 24);
```

write each character

Format specifiers

output: hello my name is M and I'm 24 years old

explanation: At the beginning of the sentence, each character is copied/written literally. Then, when the function finds a format specifier (starts with %), it will retrieve the argument

Format specifiers

%	Prints a % char
d, i	Print an int as a signed integer. %.d and %.i are synonymous for output, but are different when used with scanf for input (%.i will be interpreted as hexa if preceded by 0x or octal if by 0)
U	Print decimal unsigned int
X, x	Print an unsigned int as a hexadecimal number. x is lowercase, X is uppercase
S	Prints a null-terminated (\0) string
C	Prints a single character (Char)
P	Prints the address of a pointer or any other variable. The output is displayed in hexadecimal value. is used if we want to print void *

Return value of printf

Returns an int to check for errors

- If everything went well returns the number of bytes printed (excludes '\0')
- If an error occurred it returns a negative value

Variadic functions

A **variadic function** is a function which accepts a **variable number of arguments**. It has '...' in the function and must have at least one **mandatory argument**, if we look at the prototype of printf format is that mandatory argument

Because the rest of the arguments are variable, we use a new type of structure `va_list` and three macros from `<stdarg.h>` to handle that

`va_list` is a list that will contain all the dynamic arguments

- It's created just like any other variable type

example

```
va_list  var ;
```

↓

will end up holding all dynamic arguments

↓

```
args = ["M", 24 ]
```
→ follow up from example above

`va_start` is a macro function that initializes the `va_list` and the last

- `va_start` must be called before any use of `va_arg`
- Its purpose is to set the stage and define which elements will be stable and which will vary

```
va_start ( va_list var, parameterN );
```

`va_arg` is a macro that allows access to the arguments of the variadic function. Each time `va_arg` is called, you move to the next argument

```
va_arg ( va_list var, type_of_variable );
```

↓

int
char
(...)

example:

To access the 1st argument

va_arg(args, char *) → "M"

To access the 2nd argument

va_arg(args, int) → 24

va_end modifies the started objects so that they are no longer usable

- It frees the allocated memory

va_end(va_list var);

How to build printf ?

General formulation

1. Write each char of initial string, one by one, until % is found
2. When % is found, look at the next index where the variable defining char will be
3. Depending on what it finds it will call a function that displays the particular type of output
4. Once the 1st variable argument has been written, go back to step 1 until \0 is found